



Kubo

ERP + CRM autoalojable para PYMES
Documentación técnica y funcional

Versión 0.1.0-mvp · Armenia, Quindío · Licencia MIT
Generado el 2026-09-23

Documentación de Kubo

Documentación técnica y funcional del sistema. Está pensada para leerse en orden la primera vez y para consultarse por secciones después.

Contenido

| Documento | Para quién | Qué responde | | -- | -- | -- | | [01 – Arquitectura](01-arquitectura.md) | Técnico | ¿Cómo está construido y por qué así? Diagramas C4, flujos y atributos de calidad | | [02 – Modelo de datos](02-modelo-datos.md) | Técnico | ¿Qué se guarda, dónde y con qué reglas de integridad? | | [03 – API](03-api.md) | Desarrollador | ¿Qué endpoints existen y cómo se usan? Ejemplos con `curl` | | [04 – Seguridad](04-seguridad.md) | Técnico · Auditoría | ¿Cómo se protege la información y qué falta? | | [05 – Despliegue](05-despliegue.md) | Operación | ¿Cómo se instala, opera y respalda? | | [06 – Manual de usuario](06-manual-usuario.md) | Dueño del negocio | ¿Cómo se usa el sistema día a día? | | [07 – Pruebas](07-pruebas.md) | Calidad | ¿Qué se probó, cómo y con qué resultado? | | [08 – Trazabilidad](08-trazabilidad.md) | Evaluación | ¿Qué requisito se cumplió y dónde se comprueba? | | [09 – Guion de demostración](09-demo-guion.md) | Presentación | Guion del video de 6 minutos | | [ADRs](adr/) | Arquitectura | Las ocho decisiones que definen el sistema |

Decisiones de arquitectura (ADR)

| ADR | Decisión | | -- | -- | | [0001](adr/ADR-0001-microservicios-monolito-modular.md) | Microservicios con monolito modular interno | | [0002](adr/ADR-0002-polyrepo-contratos.md) | Polyrepo con contratos como fuente de verdad | | [0003](adr/ADR-0003-multitenancy-hibrido.md) | Multi-tenancy híbrido: `tenant\_id` hoy, RLS lista | | [0004](adr/ADR-0004-poliglotismo.md) | Poliglotismo deliberado: un lenguaje por contexto | | [0005](adr/ADR-0005-eventos-rabbitmq.md) | Eventos con RabbitMQ y publicador tolerante a fallos | | [0006](adr/ADR-0006-cifrado-campos.md) | Cifrado de campos personales con AES-256-GCM | | [0007](adr/ADR-0007-jwt-rs256-jwks.md) | JWT RS256 con JWKS: la llave privada no sale del IAM | | [0008](adr/ADR-0008-alcance-mvp.md) | Alcance declarado del MVP y recortes conscientes |

Documentación por servicio

Cada repositorio tiene su propio README con lo específico:

- [kubo-gateway](../kubo-gateway/README.md) — enrutamiento, JWT, límites de tasa
- [kubo-iam](../kubo-iam/README.md) — identidad, tokens, auditoría
- [kubo-crm](../kubo-crm/README.md) — clientes y cifrado de campos
- [kubo-erp](../kubo-erp/README.md) — catálogo, kardex, ventas y eventos

- [kubo-analytics](../kubo-analytics/README.md) — proyección e indicadores
- [kubo-web](../kubo-web/README.md) — PWA y experiencia de usuario
- [kubo-infra](../kubo-infra/README.md) — orquestación, semilla y pruebas

Generar el PDF consolidado

```
./kubo-docs/scripts/build-pdf.sh
```

Genera `Kubo-Documentacion.pdf` en la raíz del workspace, con todos los documentos y los ADR unidos. Requiere Google Chrome o Chromium instalado.

01 — Arquitectura

Kubo es un ERP + CRM autoalojable para PYMES. Se instala en el local del negocio con un solo comando y opera como aplicación web instalable (PWA), incluso sin conexión a internet.

1. Vista de contexto (C4 nivel 1)

```
flowchart LR
    DUENO["Dueño / Administrador"]
    VENDEDOR["Vendedor en mostrador"]
    KUBO["Kubo<br/>ERP + CRM autoalojable"]
    CLIENTE["Cliente del negocio<br/>(WhatsApp / correo)"]

    DUENO -->|"tablero, catálogo,<br/>clientes"| KUBO
    VENDEDOR -->|"punto de venta"| KUBO
    KUBO -->|"comprobantes y avisos<br/>(fase 2)"| CLIENTE
```

2. Vista de contenedores (C4 nivel 2)

```

flowchart TB
    subgraph NAVEGADOR["Navegador del negocio"]
        PWA["kubo-web<br/>React 19 + Vite · PWA<br/>cola offline en IndexedDB"]
    end

    subgraph SERVIDOR["Servidor del negocio (Docker Compose)"]
        GW["kubo-gateway<br/>NestJS · API Gateway<br/>JWT RS256 · límites de tasa"]
        IAM["kubo-iam<br/>Java 21 · Spring Boot 4"]
        CRM["kubo-crm<br/>Ruby 3.4 · Rails 8"]
        ERP["kubo-erp<br/>Elixir 1.17 · Phoenix 1.8"]
        ANA["kubo-analytics<br/>Python 3.13 · FastAPI"]

        PG["PostgreSQL 17<br/>kubo_iam · kubo_crm · kubo_erp"]
        MG["MongoDB 8<br/>kubo_analytics"]
        MQ["RabbitMQ 4<br/>kubo.events"]
        RD["Redis 7<br/>límites de tasa"]
    end

    PWA -->|"HTTPS /api/v1"| GW
    GW -->|"verifica JWKS"| IAM
    GW --> IAM
    GW --> CRM
    GW --> ERP
    GW --> ANA
    GW --> RD

    IAM --> PG
    CRM --> PG
    ERP --> PG
    ANA --> MG

    ERP -->|"sale.created"| MQ
    MQ --> ANA

```

Responsabilidad de cada contenedor

| Contenedor | Responsabilidad | Base de datos | | --- | --- | --- | | `kubo-gateway` | Único punto de entrada: valida el JWT contra el JWKS, aplica límites de tasa, propaga la identidad y la trazabilidad, enruta | Redis (límites) | | `kubo-iam` | Negocios, usuarios, roles, tokens, auditoría | `kubo\_iam` | | `kubo-crm` | Clientes, cifrado de datos personales, cartera | `kubo\_crm` | | `kubo-erp` | Catálogo, inventario (kardex), ventas, anulación | `kubo\_erp` | | `kubo-analytics` | Modelo de lectura de ventas, indicadores del tablero | `kubo\_analytics` (MongoDB) | | `kubo-web` | Interfaz PWA; sirve los activos y proxee `/api` al gateway | — |

3. Vista de componentes (C4 nivel 3, ejemplo: `kubo-erp`)

```

flowchart TB
    subgraph WEB["kubo-erp · capa de presentación"]
        ROUTER["Router /api/v1"]
        PLUG["Plug Identity<br/>(identidad del gateway)"]
        CTRL["ProductController<br/>SaleController<br/>StockController"]
        JSON["ProductJSON · SaleJSON"]
    end

    subgraph APP["Capa de aplicación"]
        CATALOG["Catalog<br/>catálogo e inventario"]
        SALES["Sales<br/>venta transaccional"]
        PUB["Events.Publisher<br/>(RabbitMQ)"]
        EVT["Events.SaleCreated"]
    end

    subgraph DOM["Capa de dominio"]
        PROD["Product"]
        MOVE["StockMovement"]
        SALE["Sale · SaleItem"]
    end

    subgraph INFRA["Infraestructura"]
        REPO["Ecto.Repo<br/>PostgreSQL"]
        BUS["AMQP"]
    end

    ROUTER --> PLUG --> CTRL --> JSON
    CTRL --> CATALOG
    CTRL --> SALES
    SALES --> PROD
    SALES --> SALE
    CATALOG --> PROD
    CATALOG --> MOVE
    SALES --> PUB
    PUB --> EVT
    PUB --> BUS
    CATALOG --> REPO
    SALES --> REPO

```

Los otros servicios siguen la misma separación. La regla es que el dominio no conoce HTTP, ni la base de datos, ni el bus de eventos.

4. Flujo crítico: registrar una venta

```
sequenceDiagram
    autonumber
    participant V as Vendedor (PWA)
    participant G as API Gateway
    participant E as kubo-erp
    participant P as PostgreSQL
    participant M as RabbitMQ
    participant A as kubo-analytics
    participant N as MongoDB

    V->>G: POST /api/v1/sales (Bearer JWT)
    G->>G: verifica firma RS256 contra el JWKS
    G->>E: POST /sales + X-User-Id, X-Tenant-Id
    E->>P: BEGIN
    E->>P: SELECT ... FOR UPDATE (productos)
    E->>P: INSERT sale, sale_items
    E->>P: UPDATE products.stock
    E->>P: INSERT stock_movements (kardex)
    E->>P: COMMIT
    E-->>G: 201 Created (venta)
    G-->>V: 201 Created
    E-)M: publica sale.created (asíncrono)
    M-)A: entrega del evento
    A->>N: deduplica por event_id y proyecta la venta
    Note over V,A: El tablero refleja la venta en menos de un segundo,<br/>sin que la caja
```

Si no hay internet en el local

```
sequenceDiagram
    autonumber
    participant V as Vendedor (PWA)
    participant Q as Cola IndexedDB
    participant G as API Gateway
    participant E as kubo-erp

    V->>Q: cobra sin conexión (venta guardada)
    Note over V,Q: La venta queda "pendiente de sincronizar"
    V->>V: vuelve la conexión (evento online)
    Q->>G: reintenta POST /sales con el token vigente
    G->>E: registra la venta
    E-->>Q: 201 Created
    Q->>Q: elimina la venta de la cola
```

5. Atributos de calidad y cómo se sostienen

| Atributo | Mecanismo | | --- | --- | | ****Escalabilidad**** | Servicios sin estado (excepto las bases); el gateway escala horizontalmente porque los límites de tasa viven en Redis. Cada base puede migrar a

su propia instancia sin tocar código. || ****Disponibilidad**** | Tolerancia a fallos en cadena: la venta no depende del bus ni de analítica; el gateway opera sin Redis (fail-open); cada contenedor tiene ***healthcheck*** y política de reinicio. || ****Seguridad**** | JWT asimétrico, cifrado de campos, aislamiento por negocio, auditoría encadenada, límites de tasa y validación en cada frontera. || ****Mantenibilidad**** | Un lenguaje por contexto con su ecosistema natural; arquitectura limpia dentro de cada servicio; contratos explícitos entre repos. || ****Portabilidad**** | Todo corre en Docker Compose: el mismo artefacto sirve para el mini-PC del local y para un servidor en la nube. || ****Observabilidad**** | Sonda `/health` en cada servicio con estado de sus dependencias, `correlation-id` propagado por el gateway y logs JSON estructurados. || ****Costo**** | Consumo en reposo cercano a 1.5 GB de RAM y 8 contenedores; cabe en un VPS de USD 6–12 al mes o en un equipo modesto del local. |

6. Decisiones arquitectónicas

Cada decisión relevante está registrada como ADR:

| ADR | Decisión | | --- | --- | | [0001](adr/ADR-0001-microservicios-monolito-modular.md) | Microservicios con monolito modular interno | | [0002](adr/ADR-0002-polyrepo-contratos.md) | Polyrepo con contratos como fuente de verdad | | [0003](adr/ADR-0003-multitenancy-hibrido.md) | Multi-tenancy híbrido (`tenant_id` hoy, RLS lista) | | [0004](adr/ADR-0004-poliglotismo.md) | Poliglotismo deliberado: un lenguaje por contexto | | [0005](adr/ADR-0005-eventos-rabbitmq.md) | Eventos con RabbitMQ y publicador tolerante a fallos | | [0006](adr/ADR-0006-cifrado-campos.md) | Cifrado de campos personales con AES-256-GCM | | [0007](adr/ADR-0007-jwt-rs256-jwks.md) | JWT RS256 con JWKS en el gateway | | [0008](adr/ADR-0008-alcance-mvp.md) | Alcance declarado del MVP y recortes conscientes |

7. Estructura del workspace

```
Kubo/
├─ README.md           índice y arranque rápido
├─ Makefile            up · down · seed · smoke · pdf
├─ kubo-gateway/       API Gateway (TypeScript + NestJS)
├─ kubo-iam/           identidad (Java + Spring Boot)
├─ kubo-crm/           clientes (Ruby + Rails)
├─ kubo-erp/           catálogo, inventario y ventas (Elixir + Phoenix)
├─ kubo-analytics/     tablero (Python + FastAPI + MongoDB)
├─ kubo-web/           PWA (React + Vite + Tailwind)
├─ kubo-infra/         Docker Compose, semilla y prueba de humo
└─ kubo-docs/          esta documentación
```

02 — Modelo de datos

Kubo aplica ****una base de datos por servicio****: ningún servicio consulta el

esquema de otro. Las referencias entre dominios son identificadores lógicos, no llaves foráneas.

1. Panorama

```
flowchart LR
    subgraph PG["PostgreSQL 17 · una instancia, tres bases aisladas"]
        subgraph IAMDB["kubo_iam"]
            TENANTS["tenants"]
            USERS["users"]
            REFRESH["refresh_tokens"]
            AUDIT["audit_logs"]
        end
        subgraph CRMDB["kubo_crm"]
            CUSTOMERS["customers"]
        end
        subgraph ERPDB["kubo_erp"]
            PRODUCTS["products"]
            MOVEMENTS["stock_movements"]
            SALES["sales"]
            ITEMS["sale_items"]
        end
    end

    subgraph MONGO["MongoDB 8"]
        EVENTS["events<br/>(bitácora)"]
        PROJ["sales<br/>(proyección)"]
    end

    USERS -->|"tenant_id"| TENANTS
    REFRESH -->|"user_id"| USERS
    CUSTOMERS -->|"tenant_id (lógico)"| TENANTS
    PRODUCTS -->|"tenant_id (lógico)"| TENANTS
    SALES -->|"sale_id"| ITEMS
    ITEMS -->|"product_id"| PRODUCTS
    MOVEMENTS -->|"product_id"| PRODUCTS
    SALES -->|"customer_id (lógico, sin FK)"| CUSTOMERS
    SALES -->|"evento sale.created"| EVENTS
    EVENTS -->|"proyección"| PROJ
```

2. `kubo\_iam` — identidad y auditoría

```

erDiagram
    TENANTS ||--o{ USERS : "tiene"
    USERS ||--o{ REFRESH_TOKENS : "emite"

    TENANTS {
        uuid id PK
        varchar name
        varchar slug UK
        varchar plan
        timestampz created_at
    }
    USERS {
        uuid id PK
        uuid tenant_id FK
        varchar email "único por negocio"
        varchar password_hash "BCrypt"
        varchar full_name
        varchar role "OWNER|ADMIN|SELLER|ACCOUNTANT|VIEWER"
        varchar status "ACTIVE|DISABLED"
        timestampz last_login_at
    }
    REFRESH_TOKENS {
        uuid id PK
        uuid user_id FK
        varchar token_hash UK "SHA-256, nunca el valor"
        timestampz expires_at
        timestampz revoked_at
        uuid replaced_by "rotación"
    }
    AUDIT_LOGS {
        uuid id PK
        uuid tenant_id
        uuid user_id
        varchar action
        varchar entity
        varchar entity_id
        varchar prev_hash
        varchar hash "cadena SHA-256"
        timestampz created_at
    }

```

****Decisiones del modelo****

- `refresh\_tokens.token\_hash`: si la base se filtra, los tokens no se pueden reutilizar (solo se guarda su SHA-256).
- `replaced\_by`: al rotar un token se enlaza con el que lo reemplaza. Si vuelve a usarse uno revocado, se invalida toda la familia (detección de robo).

- `audit\_logs.prev\_hash` + `hash`: cada registro encadena el anterior; alterar un registro intermedio rompe la cadena y es detectable.
- `users.email` es único **por negocio**, no global.

3. `kubo\_crm` — clientes con datos personales cifrados

```
customers
├─ id                uuid  PK
├─ tenant_id        uuid  (aislamiento por negocio)
├─ name             varchar(160)
├─ email            varchar(180)
├─ document_number_encrypted text    ← AES-256-GCM: base64(iv || tag || ciphertext)
├─ document_number_bidx varchar(64) ← HMAC-SHA256 normalizado (búsqueda exacta)
├─ phone_encrypted  text    ← AES-256-GCM
├─ phone_bidx       varchar(64) ← HMAC-SHA256
├─ city, address, notes
├─ stage            LEAD | PROSPECT | CUSTOMER
├─ credit_limit      numeric(14,2) ≥ 0
├─ deleted_at       timestampz    (borrado lógico)
└─ created_at / updated_at
```

****Índices y restricciones****

| Objeto | Propósito | | -- | -- | | `idx\_customers\_tenant\_document\_unique` (único, parcial) | Un documento no puede repetirse dentro del mismo negocio; ignora borrados | |
 `idx\_customers\_tenant\_stage` | Filtros del tablero por etapa | | `customers\_stage\_check` | Solo los tres estados previstos | | `customers\_credit\_limit\_check` | El cupo no puede ser negativo |

4. `kubo\_erp` — catálogo, inventario y ventas

erDiagram

PRODUCTS ||--o{ STOCK_MOVEMENTS : "registra"

PRODUCTS ||--o{ SALE_ITEMS : "se vende en"

SALES ||--|{ SALE_ITEMS : "contiene"

PRODUCTS {

uuid id PK

uuid tenant_id

varchar sku "único por negocio"

varchar name

numeric price "precio final con IVA"

numeric cost

numeric tax_rate "19.00 por defecto"

integer stock "proyección del kardex"

integer min_stock "alerta de reposición"

boolean active

timestampz deleted_at

}

STOCK_MOVEMENTS {

uuid id PK

uuid tenant_id

uuid product_id FK

varchar kind "IN|OUT|ADJUST"

integer quantity "> 0"

integer stock_after

varchar reason

varchar reference_type "SALE|VOID|MANUAL"

uuid reference_id

uuid created_by

timestampz inserted_at

}

SALES {

uuid id PK

uuid tenant_id

varchar number "V-000001, único por negocio"

uuid customer_id "referencia lógica al CRM"

varchar customer_name "copia histórica"

varchar status "COMPLETED|VOIDED"

varchar payment_method "CASH|CARD|TRANSFER|CREDIT"

numeric subtotal

numeric tax

numeric total

uuid sold_by

timestampz voided_at

}

SALE_ITEMS {

uuid id PK

uuid sale_id FK

uuid product_id FK

varchar product_name "copia histórica"

```

integer quantity "> 0"
numeric unit_price "copia histórica"
numeric tax_rate
numeric tax_amount
numeric total
}

```

****Reglas de integridad en el motor****

| Restricción | Qué garantiza | | -- | -- | | `products\_stock\_not\_negative` | El motor rechaza stock negativo aunque el código falle | | `stock\_movements\_reference\_unique` (único parcial) | Una venta no puede descontar dos veces el mismo producto (idempotencia) | | `sale\_items\_quantity\_positive` | No existen líneas con cantidad cero o negativa | | `sales\_total\_not\_negative` | No existen ventas con total negativo | | `products\_tenant\_sku\_unique` (único parcial) | Un SKU no se repite dentro del negocio |

****Por qué el stock es una proyección**:** la verdad está en `stock\_movements`.

`products.stock` se mantiene por velocidad de lectura y se actualiza ****dentro de**

la misma transacción****** que inserta el movimiento, con el producto bloqueado

(`SELECT ... FOR UPDATE`). Ante cualquier duda se puede reconstruir el saldo

sumando el kardex y compararlo con la proyección.

5. `kubo\_analytics` – modelo de lectura en MongoDB

```

events (bitácora cruda)
├─ _id          = event_id del evento ← índice único: deduplicación
├─ event_type   "sale.created"
├─ version      1
├─ tenant_id
├─ occurred_at  date
├─ payload      documento original completo
└─ received_at  date

sales (proyección consultable)
├─ _id          ObjectId
├─ sale_id      índice único
├─ tenant_id    + sold_at (índice compuesto)
├─ number, status, payment_method
├─ customer_id, customer_name
├─ subtotal, tax, total      (double: analítica, no contabilidad)
├─ items[]         {product_id, product_name, quantity, unit_price, tax_amount, total}
└─ sold_at, created_at, updated_at

```

****Idempotencia**:** `events.\_id` es el `event\_id`. Si el mismo evento llega dos

veces, la segunda inserción falla con clave duplicada y se descarta: una venta nunca se cuenta dos veces.

Precisión numérica: el sistema de registro contable es `kubo-erp`, con aritmética decimal. Aquí los importes se guardan como punto flotante porque las agregaciones sobre miles de documentos no necesitan precisión de centavo y así se aprovechan los operadores nativos de MongoDB. Es una decisión consciente: el tablero es para decidir, la factura es para cobrar.

6. Convenciones transversales

| Convención | Decisión | | --- | | Identificadores | UUID (v4) en todas las tablas; nunca enteros autoincrementales expuestos | | Fechas | `timestampz` en PostgreSQL, almacenadas en UTC; la interfaz muestra `America/Bogota` | | Dinero | `numeric(14,2)` y aritmética decimal en el ERP; nunca coma flotante | | Borrado | Lógico (`deleted\_at`), nunca físico: la historia comercial se conserva | | Aislamiento | `tenant\_id` en toda tabla de negocio + política RLS escrita y lista para activar | | Migraciones | Versionadas y **inmutables** una vez aplicadas; nunca se edita una migración ya ejecutada |

03 — API

Todas las peticiones entran por el **API Gateway** (`http://localhost:9080` en desarrollo). El gateway valida el JWT, aplica límites de tasa y enruta al servicio correspondiente. Ningún servicio de negocio está expuesto directamente.

- **Base**: `/api/v1`
- **Formato**: JSON UTF-8
- **Autenticación**: `Authorization: Bearer <access token>`
- **Trazabilidad**: cabecera `X-Correlation-Id` (si no llega, el gateway la crea)

1. Rutas públicas y protegidas

| Ruta | Acceso | | --- | | `GET /api/v1/health` | Público (lo atiende el gateway) | | `POST /api/v1/auth/login` | Público | | `POST /api/v1/auth/register` | Público | | `POST /api/v1/auth/refresh` | Público | | `POST /api/v1/auth/logout` | Público | | `GET /api/v1/auth/.well-known/jwks.json` | Público | | Todo lo demás | Requiere `Bearer` válido |

2. Enrutamiento

| Prefijo | Servicio | | -- | -- | | `/api/v1/auth`, `/api/v1/users`, `/api/v1/audit` | `kubo-iam` | |
`/api/v1/customers` | `kubo-crm` | | `/api/v1/products`, `/api/v1/sales`, `/api/v1/stock` | `kubo-erp` | |
`/api/v1/dashboard`, `/api/v1/events` | `kubo-analytics` |

3. Identidad (`kubo-iam`)

`POST /auth/register`

Crea un negocio y su usuario propietario.

```
curl -X POST http://localhost:9080/api/v1/auth/register \  
-H 'Content-Type: application/json' \  
-d '{  
  "tenantName": "Tienda La Esquina",  
  "fullName": "Administrador",  
  "email": "admin@kubo.local",  
  "password": "Admin123!"  
}'
```

`POST /auth/login`

```
curl -X POST http://localhost:9080/api/v1/auth/login \  
-H 'Content-Type: application/json' \  
-d '{"email": "admin@kubo.local", "password": "Admin123!"}'
```

Respuesta:

```
{  
  "accessToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6...",  
  "refreshToken": "3lP2...",  
  "tokenType": "Bearer",  
  "expiresInSeconds": 900,  
  "user": {  
    "id": "...", "email": "admin@kubo.local", "fullName": "Administrador Kubo",  
    "role": "OWNER", "tenantId": "...", "tenantName": "Tienda La Esquina"  
  }  
}
```

El **access token** dura 15 minutos y se firma con RS256. El **refresh token**

dura 7 días, se guarda hashado y **rota en cada uso**; reutilizar uno revocado

invalida toda la familia de tokens del usuario.

`GET /auth/me`

Devuelve el perfil del usuario autenticado.

`GET /auth/.well-known/jwks.json`

Llave pública RSA que el gateway usa para verificar firmas.

4. Clientes (`kubo-crm`)

| Método | Ruta | Descripción | | -- | -- | -- | | GET | `/customers?q=&stage=` | Listado (documento y teléfono ****enmascarados****) | | GET | `/customers/{id}` | Detalle (valores descifrados) | | POST | `/customers` | Crear | | PATCH | `/customers/{id}` | Actualizar | | DELETE | `/customers/{id}` | Borrado lógico | | GET | `/customers/stats` | Conteos por etapa y cartera |

```
TOKEN=$(curl -s -X POST http://localhost:9080/api/v1/auth/login \
-H 'Content-Type: application/json' \
-d '{"email":"admin@kubo.local","password":"Admin123!"}' | jq -r .accessToken)

curl -X POST http://localhost:9080/api/v1/customers \
-H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
-d '{"name":"Panadería El Trigal","document_number":"900123456-7",
  "phone":"3105551234","city":"Armenia","stage":"CUSTOMER}"'
```

5. Catálogo, inventario y ventas (`kubo-erp`)

| Método | Ruta | Descripción | | -- | -- | -- | | GET | `/products?q=&low\_stock=` | Catálogo | | POST | `/products` | Crear producto | | PATCH | `/products/{id}` | Actualizar | | DELETE | `/products/{id}` | Borrado lógico | | GET | `/products/stats` | Totales y valor del inventario | | POST | `/products/{id}/stock` | Movimiento de inventario | | GET | `/stock/movements?product\_id=` | Kardex | | GET | `/sales?status=` | Ventas | | POST | `/sales` | Registrar venta | | GET | `/sales/{id}` | Detalle | | POST | `/sales/{id}/void` | Anular venta (devuelve inventario) | | GET | `/sales/stats` | Totales del día |

Registrar una venta

```
curl -X POST http://localhost:9080/api/v1/sales \
-H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
-d '{
  "items": [
    {"product_id": "...", "quantity": 2},
    {"product_id": "...", "quantity": 1}
  ],
  "customer_id": "...",
  "customer_name": "Panadería El Trigal",
  "payment_method": "CASH"
}'
```

Respuesta `201`:

```
{
  "data": {
    "id": "...", "number": "V-000004", "status": "COMPLETED",
    "subtotal": "31680.67", "tax": "6019.33", "total": "37700.00",
    "items": [{ "product_name": "Cafe molido 250 g", "quantity": 2,
      "unit_price": "9500.00", "tax_amount": "3033.61", "total": "19000.00" }]
  }
}
```

****Reglas de negocio aplicadas****

| Situación | Respuesta | | -- | -- | | Venta sin líneas | `422 EMPTY\_SALE` | | Producto inexistente o de otro negocio | `422 PRODUCT\_NOT\_FOUND` | | Cantidad mayor al stock | `409 INSUFFICIENT\_STOCK` (con el stock disponible) | | Venta ya anulada | `409 ALREADY\_VOIDED` | | Colisión del número de venta | reintento automático (hasta 3) |

Ajustar inventario

```
curl -X POST http://localhost:9080/api/v1/products/{id}/stock \
-H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
-d '{"kind": "IN", "quantity": 40, "reason": "Compra a proveedor"}'
```

`kind` acepta `IN` (entrada), `OUT` (salida) y `ADJUST` (fijar el stock exacto).

6. Tablero (`kubo-analytics`)

| Método | Ruta | Descripción | | -- | -- | -- | | GET | `/dashboard/summary` | Ventas, ingreso, IVA, ticket promedio, unidades, hoy | | GET | `/dashboard/sales-by-day?days=14` | Serie diaria | | GET | `/dashboard/top-products?limit=10` | Productos más vendidos | | GET | `/dashboard/top-customers?limit=10` | Mejores clientes | | GET | `/dashboard/payment-methods` | Distribución por medio de pago | | GET | `/dashboard/recent-sales?limit=10` | Últimas ventas | | GET | `/dashboard/rotation` |

Productos con más salida en 7 días || POST | `/events` | Vía de respaldo para reenviar eventos por HTTP |

7. Eventos de dominio

El ERP publica en el exchange `*topic* `kubo.events``. Sobre del evento:

```
{
  "event_id": "9f1c...",
  "event_type": "sale.created",
  "version": 1,
  "occurred_at": "2026-09-23T17:50:12.480Z",
  "tenant_id": "...",
  "data": {
    "sale_id": "...", "number": "V-000004", "status": "COMPLETED",
    "customer_id": "...", "customer_name": "Panadería El Trigal",
    "payment_method": "CASH",
    "subtotal": "31680.67", "tax": "6019.33", "total": "37700.00",
    "sold_by": "...", "sold_at": "2026-09-23T17:50:12.470Z",
    "items": [
      { "product_id": "...", "product_name": "Cafe molido 250 g", "quantity": 2,
        "unit_price": "9500.00", "tax_rate": "19.00",
        "tax_amount": "3033.61", "total": "19000.00" }
    ]
  }
}
```

****Contrato de consumo****: el consumidor debe deduplicar por ``event_id`` y tolerar

campos nuevos (versionado aditivo). Los mensajes fallidos van a la cola

``kubo.analytics.sale_created.dlq``.

8. Formato de errores

Todos los servicios responden con la misma forma:

```
{
  "code": "INSUFFICIENT_STOCK",
  "message": "Stock insuficiente de Cafe molido 250 g: disponible 7"
}
```

| Código HTTP | Cuándo || -- | -- || `400` | Datos inválidos (`VALIDATION\_ERROR`,
`INVALID\_QUANTITY`) || `401` | Sin token, token inválido/expirado, identidad ausente || `404` |
Recurso inexistente o de otro negocio || `409` | Conflicto de estado (`INSUFFICIENT\_STOCK`,
`ALREADY\_VOIDED`, `EMAIL\_ALREADY\_EXISTS`) || `422` | Regla de negocio incumplida

(`EMPTY\_SALE`, `PRODUCT\_NOT\_FOUND`) || `429` | Límite de tasa excedido || `502` | Microservicio no disponible (`UPSTREAM\_UNAVAILABLE`) |

Los mensajes nunca incluyen trazas ni detalles internos; los detalles técnicos quedan en los logs con el `correlation-id` de la petición.

9. Límites de tasa

| Ámbito | Límite por defecto | | --- | --- | | Rutas de negocio | 600 peticiones/minuto por negocio (o por IP si no hay token) | | Rutas de autenticación | 40 peticiones/minuto (frena fuerza bruta) |

Las respuestas incluyen `X-RateLimit-Limit` y `X-RateLimit-Remaining`. Al excederse: `429 RATE\_LIMIT\_EXCEEDED`.

04 — Seguridad

Kubo maneja datos personales de clientes, precios de compra y cartera: la seguridad no es un añadido, es parte del diseño. Este documento explica **qué se implementó, cómo se verifica y qué queda pendiente**.

1. Modelo de confianza

```
flowchart LR
    EXT["Red del local<br/>(posible red compartida)"]
    GW["API Gateway<br/>único punto expuesto"]
    NET["Red privada de contenedores<br/>kubo-net"]
    SVC["Microservicios<br/>sin puertos públicos"]

    EXT -->|"HTTPS + JWT"| GW
    GW -->|"cabeceras de identidad<br/>X-User-Id · X-Tenant-Id"| NET
    NET --> SVC
```

- Solo el gateway (9080) y la PWA (3000) se publican. Los servicios 9081–9084 solo se mapean al host para diagnóstico local; **en producción no se publican**.
- La identidad se establece **una sola vez**, en el gateway. Los servicios internos confían en las cabeceras porque no son alcanzables desde fuera.
- El gateway **elimina cualquier cabecera `X-User-\*` que envíe el cliente** antes de inyectar la identidad verificada: sin esto, cualquiera podría

suplantar a otro usuario con un `curl`.

2. Autenticación

| Control | Implementación | | -- | -- | | Contraseñas | **BCrypt** (coste 10). Nunca se persisten ni se registran en logs | | Access token | **JWT RS256**, 15 minutos, con `iss`, `aud`, `jti`, `exp` y claims de negocio (`tenant\_id`, `role`) | | Verificación | Solo el gateway, contra el **JWKS** del servicio de identidad, con caché y refresco automático | | Refresh token | Valor aleatorio de 384 bits; se guarda **solo su SHA-256** | | Rotación | Cada refresh emite uno nuevo y revoca el anterior | | Detección de reuso | Si se usa un token ya rotado, se **invalida toda la familia** del usuario y se audita `REFRESH\_REUSE\_DETECTED` | | Cierre de sesión | Revoca el refresh token en el servidor, no solo en el navegador |

Por qué asimétrico: con un secreto compartido, comprometer cualquier servicio

permitiría *emitir* tokens válidos para cualquier usuario. Con RS256, la llave

privada vive solo en `kubo-iam`.

3. Cifrado

| Ámbito | Mecanismo | Dónde se verifica | | -- | -- | -- | | En tránsito (externo) | TLS 1.3 terminado en el proxy inverso | Configuración de despliegue | | En tránsito (interno) | Red privada de contenedores; mTLS previsto para clúster | - | | **Datos personales** | **AES-256-GCM** campo a campo (documento, teléfono) | `make smoke` inspecciona el texto cifrado en PostgreSQL | | Búsqueda sobre cifrado | **Índice ciego** HMAC-SHA256 normalizado | `make smoke` verifica que el índice no contiene el valor | | Contraseñas | BCrypt con sal por usuario | - | | Respaldos | Cifrado con `age` antes de salir del servidor | Fase 2 |

Formato almacenado: `base64(iv || tag || ciphertext)`. El **tag de GCM** hace que

cualquier manipulación del texto cifrado sea detectable: el descifrado falla y el

servicio devuelve `null` en lugar de un valor corrupto.

Gestión de claves

- Las claves llegan por variable de entorno y **nunca** están en el repositorio

(`.env` está en `.gitignore`; solo se versiona `.env.example` con ceros).

- `KUBO\_FIELD\_ENCRYPTION\_KEY` y `KUBO\_BLIND\_INDEX\_KEY` son de 32 bytes en hex;

el servicio **falla al arrancar** si no cumplen el formato, en lugar de cifrar

con una clave débil.

- El servicio **rechaza explícitamente** las claves de ejemplo de ``env.example``: desplegar copiando la plantilla sin generar claves produce un error claro (``«conserva el valor de ejemplo; genere una clave real»``) en vez de proteger los datos con una clave conocida por cualquiera. Cubierto por dos pruebas unitarias.
- La llave de firma JWT puede llegar como PEM; si no, el servicio genera un par **efímero** en memoria y lo advierte en los logs (solo desarrollo).
- **Pendiente**: envoltura KEK/DEK por negocio con KMS para la versión en nube, y rotación de claves de campo (exige re-cifrado por lotes).

4. Aislamiento entre negocios

1. Toda tabla de negocio lleva ``tenant_id`` y toda consulta lo filtra.
2. El ``tenant_id`` **nunca** viene del cuerpo ni de un parámetro: se toma del claim del JWT, que el gateway propaga como cabecera.
3. La política de *row level security* está escrita y versionada en cada servicio (``db/rls/enable-rls.sql``) lista para activarse como segunda barrera.
4. **Verificación automática**: ``make smoke`` registra un segundo negocio y comprueba que no ve ni un cliente ni un producto del primero.

5. Autorización

Roles definidos: ``OWNER``, ``ADMIN``, ``SELLER``, ``ACCOUNTANT``, ``VIEWER``. El rol viaja en el token y el gateway lo propaga; cada servicio decide qué permite. En el MVP el control es por rol y negocio (no hay permisos finos por recurso), lo que se documenta como alcance.

6. Auditoría

``kubo-iam`` mantiene ``audit_logs`` **append-only** con cadena de hash: cada registro incluye el hash del anterior, de modo que alterar o borrar un registro intermedio

rompe la cadena y es detectable. Se auditan, entre otros: registro de negocio, inicio de sesión correcto y fallido, rotación de token, reuso detectado.

Los logs son **JSON estructurados** y el gateway **redacta** la cabecera

``Authorization`` y las cookies. Nunca se registran contraseñas, tokens ni datos

personales: Rails filtra ``document_number``, ``phone`` y ``email`` en sus logs.

7. Protección de la interfaz y la API

| Amenaza | Mitigación | | --- | --- | | Fuerza bruta de contraseñas | Límite de 40 peticiones/minuto en rutas de autenticación + auditoría de intentos fallidos | | Abuso de la API | Límite de 600 peticiones/minuto por negocio o IP | | XSS | React escapa por defecto; no se usa ``dangerouslySetInnerHTML``; cabeceras de seguridad con Helmet | | Clickjacking | ``X-Frame-Options`` de Helmet | | MIME sniffing | ``X-Content-Type-Options: nosniff`` | | Inyección SQL | Consultas parametrizadas en los cuatro lenguajes (JPA, ActiveRecord, Ecto, PyMongo); **no hay SQL concatenado** | | Inyección en búsquedas | Los comodines ``%`` y ``_`` se eliminan del término de búsqueda | | CSRF | La API no usa cookies de sesión: la autenticación es por cabecera ``Authorization`` | | Enumeración de usuarios | El login responde el mismo mensaje para correo inexistente y contraseña incorrecta | | Fuga de datos entre negocios | Aislamiento por ``tenant_id`` + prueba automática |

Sobre el almacenamiento del refresh token

El refresh token se guarda en ``localStorage`` del navegador. Es una decisión

consciente y **documentada como riesgo**: un XSS exitoso podría robarlo. Las

mitigaciones aplicadas son: el access token vive solo en memoria, el refresh

rota en cada uso y el reuso se detecta invalidando la familia completa. La

solución definitiva (cookie ``httpOnly`` + ``SameSite=Strict`` con BFF) está prevista

para la fase 2.

8. Cumplimiento normativo

| Norma | Cómo se aborda | | --- | --- | | **Ley 1581 de 2012** (protección de datos personales, Colombia) | Cifrado de datos personales, minimización (enmascarado en listados), borrado lógico reversible, trazabilidad de accesos. **Pendiente**: registro de autorizaciones del titular, política de retención y procedimiento de supresión definitiva | | **Facturación electrónica DIAN** | Fuera del alcance del MVP (requiere ser Proveedor Tecnológico autorizado). El dominio deja preparado el puerto de facturación para emitir UBL 2.1 con CUFE y QR en la fase 3 | | **Habeas data** en la

interfaz** | El manual de usuario documenta qué datos se guardan, para qué y cómo se solicitan al cliente |

9. Verificación

```
make smoke
```

La prueba de humo comprueba, entre otras cosas:

- Petición sin token → `401`.
- Token con firma falsificada → `401`.
- Cabecera `X-User-Id` inyectada por el cliente → **ignorada** (`401`).
- Documento cifrado en PostgreSQL y legible solo por el detalle autorizado.
- Listado con el documento enmascarado.
- Índice ciego sin el valor en claro.
- Segundo negocio que no ve datos del primero.

10. Pendientes de seguridad (fase 2)

| Pendiente | Riesgo que cierra | | --- | --- | | Refresh token en cookie `httpOnly` + `SameSite` | Robo de token por XSS | | Activación de RLS con interceptor de transacción | Error humano en un filtro de consulta | | mTLS entre gateway y servicios | Movimiento lateral dentro del clúster | | Rotación de claves de cifrado de campo | Compromiso de una clave a largo plazo | | Segundo factor (TOTP) para el propietario | Robo de credenciales | | Análisis SAST/SCA automatizado en CI y escaneo de secretos | Dependencias vulnerables y credenciales filtradas | | Rate limiting por usuario (no solo por negocio) | Abuso desde una cuenta comprometida |

05 — Despliegue y operación

1. Requisitos

| Recurso | Mínimo | Recomendado | | --- | --- | --- | | CPU | 2 núcleos | 4 núcleos | | RAM | 4 GB | 8 GB | | Disco | 20 GB | 60 GB SSD | | Sistema | Linux con Docker Engine 24+ y el plugin `docker compose` | Ubuntu 24.04 LTS | | Red | Salida a internet solo para construir/actualizar | — |

El sistema completo consume **menos de 2 GB de RAM en reposo** (límites por contenedor definidos en `kubo-infra/docker-compose.yml`).

2. Instalación en el local del negocio

```
git clone <url-del-workspace> kubo && cd kubo
cp kubo-infra/.env.example kubo-infra/.env
```

Editar `kubo-infra/.env` y **cambiar todos los valores**. Generar las claves:

```
openssl rand -hex 32 # KUBO_FIELD_ENCRYPTION_KEY
openssl rand -hex 32 # KUBO_BLIND_INDEX_KEY
openssl rand -hex 64 # KUBO_CRM_SECRET_KEY_BASE
openssl rand -base64 48 # KUBO_ERP_SECRET_KEY_BASE
```

Levantar:

```
make up # construye y arranca los 8 contenedores
make seed # datos de demostración (omitir en producción)
make smoke # verificación end-to-end
```

Abrir `http://<ip-del-servidor>:3000` e ingresar con el administrador creado.

Puertos

Servicio	Host	Contenedor	---	---	---	PWA	3000	80	API Gateway	9080	8080	IAM	CRM	ERP	Analítica	9081–9084	8081–8084	PostgreSQL	MongoDB	5433	27018	5432	27017	RabbitMQ (AMQP)	consola	5673	15673	5672	15672	Redis	6380	6379
----------	------	------------	-----	-----	-----	-----	------	----	-------------	------	------	-----	-----	-----	-----------	-----------	-----------	------------	---------	------	-------	------	-------	-----------------	---------	------	-------	------	-------	-------	------	------

Los puertos se eligieron **fuera del rango 8000–8100** para no chocar con otros

servicios que ya corran en la máquina del negocio.

3. Puesta en producción (checklist)

- [] `kubo-infra/.env` con todas las claves cambiadas y el archivo **fuera del control de versiones**.
- [] `KUBO\_JWT\_PRIVATE\_KEY` con una llave RSA real (si queda vacía, el servicio genera una efímera y los tokens se invalidan al reiniciar).
- [] TLS terminado en un proxy inverso (Caddy, Traefik o nginx) con certificado válido.
- [] **No publicar** los puertos 9081–9084 en producción: los servicios de negocio solo deben ser alcanzables por el gateway dentro de la red de contenedores.
- [] `KUBO\_SEED\_ENABLED=false` (no sembrar datos de demostración).
- [] RespalDOS programados (sección 5).

- [] Monitoreo de las sondas `/api/v1/health` de los cinco servicios.

TLS con Caddy (ejemplo)

```
kubo.minegocio.co {  
  reverse_proxy localhost:3000  
}
```

La PWA sirve los activos y proxee `/api` al gateway, así que un único dominio

con HTTPS cubre todo el sistema.

4. Operación diaria

| Comando | Para qué | | --- | | --- | | `make ps` | Ver el estado y la salud de cada contenedor | | `make logs` | Logs en vivo de todos los servicios | | `make restart` | Reiniciar los servicios de aplicación | | `make smoke` | Verificar el flujo completo (útil tras una actualización) | | `make down` | Detener el sistema conservando los datos | | `make clean` | Detener **y borrar** los volúmenes (pierde los datos) |

Actualización

```
git pull  
make up      # reconstruye solo lo que cambió  
make smoke   # confirma que todo sigue funcionando
```

Las migraciones se aplican solas al arrancar cada servicio

(`KUBO\_RUN\_MIGRATIONS=true`): Flyway en IAM, `db:prepare` en Rails y

`ecto.migrate` en Phoenix. Una migración aplicada **nunca se edita**; se crea una

nueva.

5. Respaldos

| Qué | Cómo | Frecuencia | | --- | | --- | | PostgreSQL | `pg\_dump` por base (`kubo\_iam`, `kubo\_crm`, `kubo\_erp`) | Diaria | | MongoDB | `mongodump` de `kubo\_analytics` | Diaria | | Configuración | `kubo-infra/.env` (cifrado con `age`) | En cada cambio | | Volúmenes Docker | `docker run --rm -v kubo\_pgdata:/data ...` | Semanal |

```
# Respaldo completo
docker exec kubo-postgres pg_dump -U kubo_root -d kubo_iam > respaldo_iam.sql
docker exec kubo-postgres pg_dump -U kubo_root -d kubo_crm > respaldo_crm.sql
docker exec kubo-postgres pg_dump -U kubo_root -d kubo_erp > respaldo_erp.sql
docker exec kubo-mongo mongodump --db kubo_analytics --archive > respaldo_analytics.archive

# Restauración
docker exec -i kubo-postgres psql -U kubo_root -d kubo_erp < respaldo_erp.sql
docker exec -i kubo-mongo mongorestore --archive --drop < respaldo_analytics.archive
```

****Objetivos**:** RPO 24 h · RTO 4 h. Un respaldo que nunca se restauró no es un

respaldo: haz un simulacro mensual contra un entorno de prueba.

****Importante**:** MongoDB guarda una ****proyección**** reconstruible desde los eventos

del ERP. PostgreSQL es la fuente de verdad y es lo que no puede perderse.

6. Diagnóstico rápido

| Síntoma | Primera revisión | | -- | -- | | La PWA carga pero no hay datos | `make ps` — ¿el gateway está **healthy**? | | `401` en todas las llamadas | El gateway no puede leer el JWKS: revisa `kubo-iam` | | `502 UPSTREAM\_UNAVAILABLE` | El microservicio destino está caído: `docker logs kubo-erp` | | El tablero no refleja ventas | `docker logs kubo-analytics` — ¿el consumidor está conectado? | | `429` en operación normal | Ajusta `KUBO\_RATE\_LIMIT\_PER\_MINUTE` en el gateway | | El inventario no cuadra | Compara `products.stock` con la suma del kardex (consulta en `02-modelo-datos.md`) |

7. Escalado

El sistema está diseñado para crecer sin reescribir:

1. ****Más memoria/CPU****: subir los `mem\_limit` del compose.
2. ****Más carga en la caja****: escalar el ERP horizontalmente (`docker compose up -d --scale kubo-erp=3`) porque es sin estado; el gateway reparte.
3. ****Bases separadas****: mover cada base a su propia instancia cambiando solo la URL de conexión.
4. ****Clúster****: los mismos artefactos corren en Kubernetes; falta mTLS entre servicios y la activación de RLS (fase 2).

8. Lo que falta para producción real

| Pendiente | Impacto | | -- | -- | | **Transactional outbox** | Un evento puede perderse si el proceso muere justo tras el `commit` | | Activación de RLS | El aislamiento depende hoy de la disciplina del

código | | Refresh token en cookie `httpOnly` | Un XSS podría robar el token de refresco | | Recuperación de contraseña | Hoy la cambia un administrador desde la base | | CI/CD y escaneos automatizados | Las verificaciones son manuales (`make smoke`) | | Monitoreo centralizado | Solo hay sondas de salud y logs locales |

06 — Manual de usuario

Guía para el dueño del negocio y sus vendedores. No requiere conocimientos técnicos.

1. Ingresar

1. Abre el navegador en la dirección que te entregó el instalador

(por ejemplo `http://192.168.1.50:3000`).

2. Escribe tu correo y contraseña.

3. Presiona **Ingresar**.

Instala Kubo como aplicación (recomendado): en el navegador busca el ícono de instalar (o el menú : → **Instalar aplicación**). Queda un acceso directo como cualquier otro programa y abre a pantalla completa.

> La primera vez recibirás el usuario administrador. Cámbialo apenas puedas.

2. El tablero

Es la primera pantalla y responde una pregunta: **¿cómo va el negocio hoy?**

| Indicador | Qué significa | | --- | --- | | **Ventas de hoy** | Cuánto se vendió hoy y cuántas transacciones | | **Ingreso acumulado** | Total vendido desde que usas Kubo | | **Ticket promedio** | Lo que gasta un cliente en promedio | | **Unidades vendidas** | Cuántos productos salieron | | **Ventas de los últimos 14 días** | La tendencia: ¿sube o baja? | | **Medios de pago** | Cuánto entra en efectivo, tarjeta o transferencia | | **Productos más vendidos** | En qué se está yendo la mercancía | | **Últimas ventas** | El detalle reciente, venta por venta |

Si el local se queda sin internet, verás la etiqueta **Sin internet**: el

tablero muestra lo último que se sincronizó y puedes seguir vendiendo.

3. Vender (punto de venta)

1. Entra a **Vender**.
2. Toca los productos para agregarlos al carrito (usa el buscador si tienes muchos).
3. Ajusta cantidades con **++** y **--**.
4. Elige el **cliente** (o deja **Consumidor final**).
5. Elige el **medio de pago**: efectivo, tarjeta, transferencia o crédito.
6. Presiona **Cobrar**.

Lo que ocurre al cobrar:

- Se descuenta el inventario en el mismo instante.
- Se asigna un número de venta (V-000001, V-000002...).
- El tablero se actualiza solo, en segundos.

Vender sin internet

Si se cae la conexión, Kubo **no te deja tirado**: la venta se guarda en el navegador y aparece la etiqueta **«1 venta por sincronizar»**. Cuando vuelva la conexión:

- La aplicación la envía automáticamente, o
 - Presiona **Sincronizar** en la barra superior.
- > Mientras no haya conexión no podrás consultar el stock más reciente de otros
> equipos, pero las ventas nunca se pierden.

4. Productos e inventario

En **Productos** ves el catálogo con precio, IVA y existencias.

- **Nuevo producto**: pide código (SKU), nombre, precio de venta, costo, tarifa de IVA y stock mínimo.
- El **precio de venta** es el precio final que paga el cliente (con IVA incluido). Kubo desagrega el impuesto al facturar.

- El **stock mínimo** dispara la alerta de reposición.

- **Ajustar inventario** (ícono de cajas): registra entradas por compra, salidas por merma o conteos físicos.

| Tipo | Cuándo usarlo |

| --- | --- |

| **Entrada (compra)** | Llegó mercancía del proveedor |

| **Salida (merma, consumo)** | Se dañó, se venció o se usó internamente |

| **Fijar stock exacto** | Después de un conteo físico |

Escribe siempre el **motivo**: queda en el kardex con fecha y usuario.

- **Archivar** (papelera): el producto deja de venderse pero su historia se conserva. Nunca se borra nada de verdad.

Cuando algo está en o por debajo del mínimo, aparece un aviso amarillo arriba:
es el momento de programar una compra.

5. Clientes

En **Clientes** está tu cartera.

- **Nuevo cliente**: nombre o razón social, documento (NIT o cédula), teléfono, correo, ciudad, dirección, etapa y cupo de crédito.

- **Etapas**: *Prospecto nuevo* → *En negociación* → *Cliente*. Sirve para saber a quién hay que llamar.

- El **documento y el teléfono se guardan cifrados**: si alguien se lleva una copia de la base de datos, no puede leerlos.

- En la lista solo se ven los últimos dígitos (`\*\*\*\*\*432`); el dato completo aparece al abrir el cliente.

> **Protección de datos**: Kubo guarda los datos personales únicamente para

> gestionar la relación comercial. Informa a tus clientes qué guardas y para

> qué, y atiende sus solicitudes de corrección o eliminación.

6. Preguntas frecuentes

****¿Necesito internet para vender?****

No. Solo para sincronizar; las ventas se guardan localmente.

****¿Puedo tener varias cajas al mismo tiempo?****

Sí. Cada equipo entra con su propio usuario. El stock se descuenta en el servidor, así que dos cajas no pueden vender la misma unidad.

****¿Dónde están mis datos?****

En el equipo donde se instaló Kubo, no en la nube de terceros.

****¿Cuánto cuesta?****

El software es tuyo (licencia MIT). Solo pagas el equipo donde corre.

****Me equivoqué en una venta.****

Pídele al administrador que la ****anule****: el inventario vuelve y queda registrada la anulación (no se borra la historia).

****¿Puedo usar mi lector de código de barras?****

Sí, si escribes el código como texto. Escribe o escanea en el buscador del POS.

****Olvidé mi contraseña.****

Pídele al administrador que la restablezca (la recuperación por correo llega en la siguiente versión).

7. Buenas prácticas

1. ****Registra el costo**** de cada producto: es lo que permite saber tu margen.

2. ****Haz un conteo físico**** una vez al mes y ajusta el stock con el motivo

«Conteo físico»: así el kardex cuenta la historia real.

3. ****Usa las etapas de cliente****: es tu lista de trabajo comercial.

4. ****Revisa el tablero cada mañana****: cinco segundos para saber cómo arrancó el día.

5. **No compartas el usuario administrador**: crea un usuario por vendedor.

07 — Pruebas

La verificación del sistema tiene tres niveles: pruebas unitarias dentro de cada servicio, una prueba de humo end-to-end y verificaciones manuales de operación.

1. Prueba de humo end-to-end (la principal)

```
make smoke
```

Ejecuta **33 comprobaciones** contra el sistema en ejecución, usando el API

Gateway como un cliente real. Resultado esperado: `33 pruebas exitosas, 0 fallidas`.

| Bloque | Qué verifica | Comprobaciones | | --- | --- | | 1. Salud | Los cinco servicios de aplicación responden `UP` | 5 | | 2. Autenticación | Login devuelve token; el token es **RS256**; trae el negocio; el **JWKS** publica la llave | 4 | | 3. Control de acceso | Sin token → `401`; token falsificado → `401`; cabecera `X-User-Id` inyectada → **ignorada** | 3 | | 4. Clientes y cifrado | Crear cliente; detalle con documento completo; listado **enmascarado**; texto **cifrado** en PostgreSQL; índice ciego correcto | 5 | | 5. Inventario y venta | Crear producto; entrada deja 10 unidades; venta por `35700.00`; IVA desagregado `5700.00`; stock baja a 7; **kardex** con 2 movimientos; sobreventa → `409` | 8 | | 6. Evento y tablero | La venta llega al modelo de lectura; MongoDB guarda el evento; el tablero la reporta | 3 | | 7. Anulación | La venta queda `VOIDED`; el inventario vuelve a 10 | 2 | | 8. Aislamiento | Un segundo negocio no ve clientes ni catálogo del primero | 3 |

La prueba es **idempotente**: crea sus propios datos con marcas de tiempo y puede ejecutarse tantas veces como haga falta.

2. Pruebas unitarias por servicio

```
# kubo-iam – firma de tokens y JWKS (3 pruebas)
cd kubo-iam && mvn test

# kubo-gateway – tabla de rutas y rutas públicas (3 pruebas)
cd kubo-gateway && npm test

# kubo-crm – cifrado de campos (6 pruebas, sin base de datos)
cd kubo-crm && ruby test/field_cipher_test.rb

# kubo-erp – aritmética de dinero (4 pruebas, sin base de datos)
cd kubo-erp && mix test test/kubo_erp/sales_totals_test.exs

# kubo-analytics – conversión de eventos (6 pruebas, sin MongoDB)
cd kubo-analytics && pip install -r requirements-dev.txt && pytest -q
```

****Total: 22 pruebas unitarias**** más 33 comprobaciones end-to-end.

Qué cubren las pruebas unitarias

| Servicio | Casos | | -- | -- | | IAM | El token se firma con RS256 y tiene 3 segmentos; el JWKS ****no**** expone la llave privada; los refresh tokens son aleatorios y su hash es SHA-256 hex | | Gateway | La tabla cubre los cuatro microservicios; los prefijos anidados resuelven al servicio correcto; una ruta protegida no se confunde con una pública | | CRM | Ida y vuelta del cifrado; dos cifrados del mismo valor difieren (IV aleatorio); ****GCM detecta manipulación****; el índice ciego es determinista y normalizado; enmascarado correcto; valores vacíos no se cifran | | ERP | IVA desagregado de un precio con impuesto incluido; suma de varias líneas; producto exento; el total siempre es subtotal + impuesto | | Analítica | Conversión de importes y cantidades desde el evento; tolerancia a entradas inválidas; normalización del detalle; fechas a UTC |

3. Verificación manual de la interfaz

| # | Paso | Resultado esperado | | -- | -- | -- | | 1 | Abrir `http://localhost:3000` | Pantalla de ingreso con la marca Kubo | | 2 | Ingresar con `admin@kubo.local` / `Admin123!` | Tablero con indicadores | | 3 | Instalar la aplicación desde el navegador | Se abre a pantalla completa, sin barra del navegador | | 4 | Registrar una venta en el POS | Aparece el número de venta y el tablero se actualiza | | 5 | Activar el modo avión y vender | Etiqueta «Sin internet» y la venta queda en cola | | 6 | Recuperar la conexión | La venta se sincroniza y el contador desaparece | | 7 | Abrir un cliente | Documento completo; en el listado, enmascarado | | 8 | Registrar una entrada de inventario | El stock sube y el movimiento queda en el kardex | | 9 | Intentar vender más unidades que el stock | Mensaje «Stock insuficiente» con la cantidad disponible |

4. Verificación de seguridad

```
# Sin token
curl -o /dev/null -w '%{http_code}\n' http://localhost:9080/api/v1/customers # 401

# Suplantación por cabecera
curl -o /dev/null -w '%{http_code}\n' http://localhost:9080/api/v1/customers \
  -H 'X-User-Id: 00000000-0000-0000-0000-000000000000' # 401

# El texto cifrado no contiene el documento
docker exec kubo-postgres psql -U kubo_root -d kubo_crm -c \
  "select document_number_encrypted from customers limit 1"

# La llave privada no se publica
curl -s http://localhost:9080/api/v1/auth/.well-known/jwks.json | grep -c '"d"' # 0

# Límite de tasa en autenticación
for i in $(seq 1 50); do
  curl -s -o /dev/null -w '%{http_code} ' -X POST http://localhost:9080/api/v1/auth/login \
    -H 'Content-Type: application/json' -d '{"email":"x@x.co","password":"malo"}'
done; echo # inc
```

5. Rendimiento observado

Medido en un equipo de 16 núcleos con 4 GB disponibles para el stack:

| Operación | Tiempo | | --- | --- | | Login (BCrypt + firma RSA) | ~180 ms | | Registrar venta (transacción + kardex) | ~45 ms | | Consultar catálogo (300 productos) | ~20 ms | | Tablero completo (5 agregaciones) | ~60 ms | | Propagación del evento a MongoDB | < 1 s | | Arranque completo del sistema | ~50 s | | Consumo en reposo | ~1.6 GB |

6. Pruebas pendientes (fase 2)

| Prueba | Por qué falta | | --- | --- | | Integración con base de datos real por servicio | Requiere Testcontainers por lenguaje | | Contratos (Pact/OpenAPI) automatizados | Hoy el contrato se verifica en la prueba de humo | | Carga (k6): 50 cajas simultáneas | Falta escenario de estrés | | E2E de navegador (Playwright) | Hoy la interfaz se verifica manualmente | | Escaneo SAST/SCA y de secretos en CI | Las verificaciones son manuales | | Simulacro de restauración de respaldo | Documentado, no ejecutado | | Accesibilidad automatizada (axe) | Solo revisión manual de contraste y foco |

08 — Matriz de trazabilidad

Cada requisito del enunciado, cómo se implementó y ****dónde se comprueba****. Si un

requisito no tiene evidencia verificable, aparece marcado como pendiente.

1. Requisitos del proyecto

| # | Requisito | Implementación | Evidencia | Estado | | --- | --- | --- | --- | --- | | R1 | ERP + CRM para PYMES sin acceso a tecnología, sin pagar SaaS | Instalación de un comando en el local, licencia MIT, sin dependencias de nube | `README.md` · `05-despliegue.md` · `make up` | Cumplido | | R2 | Mínimo 4 microservicios | 5 servicios: IAM, CRM, ERP, Analítica y Documentos (planificado) | `docker compose ps` (9 contenedores) · `01-arquitectura.md` | Cumplido (4 desplegados) | | R3 | Mínimo 3 tecnologías distintas | Java, Ruby, Elixir, Python, TypeScript (5) | `01-arquitectura.md` · ADR-0004 | Cumplido | | R4 | Front dinámico y actual, nivel senior | React 19 + Vite 8 + Tailwind 4, design system con tokens, TanStack Query, gráficas, accesibilidad | `kubo-web/` · `kubo-web/src/components/ui.tsx` | Cumplido | | R5 | Conexión con PostgreSQL y MongoDB | PostgreSQL 17 (IAM, CRM, ERP) + MongoDB 8 (analítica) | `docker compose ps` · `02-modelo-datos.md` | Cumplido | | R6 | Interfaz UI/UX | 5 pantallas, estados de carga/vacío/error, toasts, modales accesibles, español, modo offline visible | `06-manual-usuario.md` · verificación manual en `07-pruebas.md` §3 | Cumplido | | R7 | API Gateway | `kubo-gateway` (NestJS): validación JWT, límites de tasa, correlación, enrutamiento | `kubo-gateway/README.md` · `make smoke` bloque 3 | Cumplido | | R8 | Cada microservicio con su base de datos | `database-per-service`: `kubo\_iam`, `kubo\_crm`, `kubo\_erp` con roles aislados; `kubo\_analytics` en MongoDB | `kubo-infra/scripts/init-db.sh` · ADR-0003 | Cumplido | | R9 | Repositorio separado por parte del software | 9 repositorios: 5 componentes + infra + docs + web + raíz | `ls -d kubo-\*/` · ADR-0002 | Cumplido | | R10 | Documentación detallada del funcionamiento | 9 documentos + 8 ADRs + README por servicio + OpenAPI + guion de video | `kubo-docs/` · `Kubo-Documentacion.pdf` | Cumplido | | R11 | Encriptado de datos | AES-256-GCM campo a campo + índice ciego HMAC + BCrypt + SHA-256 de refresh tokens + auditoría con cadena de hash | `make smoke` bloque 4 · `04-seguridad.md` | Cumplido | | R12 | PWA | Service worker, manifest, instalable, cola offline en IndexedDB, sincronización automática | `curl localhost:3000/manifest.webmanifest` · `kubo-web/vite.config.ts` | Cumplido | | R13 | JWT | RS256 con JWKS, access 15 min, refresh rotativo con detección de reuso | `make smoke` bloque 2 · ADR-0007 | Cumplido | | R14 | Arquitectura idónea al tipo de proyecto | Microservicios con monolito modular, polyrepo, eventos, multi-tenancy híbrido | `01-arquitectura.md` · ADR-0001 a 0008 | Cumplido | | R15 | Diseño y seguridad del proyecto | Modelo de confianza, STRIDE por servicio, OWASP, Ley 1581, límites de tasa, anti-suplantación | `04-seguridad.md` · `make smoke` bloque 3 y 8 | Cumplido |

2. Requisitos funcionales del dominio

| # | Requisito | Implementación | Evidencia | | --- | --- | --- | --- | | RF-01 | Autenticar usuarios y aislar negocios | `kubo-iam` + `tenant\_id` en el token | `make smoke` bloques 2 y 8 | | RF-02 | Gestionar clientes con datos personales protegidos | `kubo-crm` con cifrado de campo | `make smoke` bloque 4 | | RF-03 | Gestionar catálogo de productos | `kubo-erp` CRUD + borrado lógico | `make smoke` bloque 5 | | RF-04 | Controlar inventario con trazabilidad | Kardex `stock\_movements` + proyección de stock | `make smoke` bloque 5 y 7 | | RF-05 | Registrar ventas con descuento de inventario | Transacción con bloqueo pesimista | `make smoke` bloque 5 | | RF-06 | Anular ventas devolviendo el

inventario | `POST /sales/:id/void` | `make smoke` bloque 7 || RF-07 | Calcular impuestos (IVA) | Desagregación con `Decimal` en el ERP | `make smoke` bloque 5 || RF-08 | Tablero de indicadores del negocio | `kubo-analytics` sobre MongoDB | `make smoke` bloque 6 || RF-09 | Operar sin conexión | Cola IndexedDB + sincronización | Verificación manual `07-pruebas.md` §3 || RF-10 | Auditar las acciones sensibles | `audit\_logs` con cadena de hash | `GET /api/v1/audit` |

3. Requisitos no funcionales

| # | Atributo | Objetivo | Cómo se sostiene | Evidencia | | --- | --- | --- | --- | --- | | RNF-01 | Rendimiento | Venta < 100 ms | Transacción única + Elixir | `07-pruebas.md` §5 || RNF-02 | Disponibilidad | El bus caído no bloquea la caja | Publicación asíncrona tolerante a fallos | ADR-0005 || RNF-03 | Escalabilidad | Escalar la caja sin reescribir | Servicios sin estado | `05-despliegue.md` §7 || RNF-04 | Seguridad | Sin datos personales legibles en la base | AES-256-GCM + índice ciego | `make smoke` bloque 4 || RNF-05 | Mantenibilidad | Un lenguaje por contexto con arquitectura limpia | ADR-0004 · README por servicio || RNF-06 | Portabilidad | Mismo artefacto en local y nube | Docker Compose | `05-despliegue.md` || RNF-07 | Consumo | < 2 GB de RAM en reposo | Límites por contenedor | `docker stats --no-stream` || RNF-08 | Usabilidad | Dueño sin conocimientos técnicos | Manual + modo simple + español | `06-manual-usuario.md` || RNF-09 | Observabilidad | Estado de cada servicio y su dependencia | Sondas `/health` | `make smoke` bloque 1 || RNF-10 | Trazabilidad | Reconstruir una operación completa | `X-Correlation-Id` propagado | Logs del gateway |

4. Decisiones de arquitectura

| ADR | Decisión | Estado | | --- | --- | --- | | 0001 | Microservicios con monolito modular interno | Aceptada | | 0002 | Polyrepo con contratos como fuente de verdad | Aceptada | | 0003 | Multi-tenancy híbrido (`tenant\_id` hoy, RLS lista) | Aceptada | | 0004 | Poliglotismo deliberado: un lenguaje por contexto | Aceptada | | 0005 | Eventos con RabbitMQ y publicador tolerante a fallos | Aceptada (outbox diferido) | | 0006 | Cifrado de campos personales con AES-256-GCM | Aceptada | | 0007 | JWT RS256 con JWKS en el gateway | Aceptada | | 0008 | Alcance declarado del MVP y recortes conscientes | Aceptada |

5. Requisitos no cubiertos (declarados)

| Requisito | Motivo | Ruta | | --- | --- | --- | | Facturación electrónica DIAN (UBL 2.1, CUF, QR) | Requiere ser Proveedor Tecnológico autorizado | Fase 3 | | Multi-rubro (Vertical Packs) | El MVP demuestra un vertical completo | Fase 3 | | *Transactional outbox* | Requiere tabla de salida y publicador de barrido | Fase 2 | | Activación de RLS | Requiere interceptor de transacción probado | Fase 2 | | Refresh token en cookie `httpOnly` | Exige BFF con cookies | Fase 2 | | Recuperación de contraseña | Requiere proveedor de correo | Fase 2 | | App móvil nativa | La PWA cubre el caso de uso | Fase 4 | | Kubernetes y CI/CD automatizado | El objetivo es un local con Docker Compose | Fase 4 |

> Ningún requisito se declara cumplido sin evidencia verificable. Los recortes

> están documentados en el ADR-0008 y en esta matriz.

09 — Guion de demostración (video)

****Duración objetivo: 6 a 7 minutos.**** El guion está escrito para grabarse en una sola toma, sin edición, con el sistema ya levantado.

Preparación (antes de grabar)

```
cd "Documents/Proyectos de Programación/Kubo"
make foreign-stop      # libera RAM (detiene Dolibarr, Guacamole, Open Notebook)
make up                # levanta los 8 contenedores
make seed              # carga catálogo, clientes y ventas de ejemplo
make smoke             # verifica 33 comprobaciones end-to-end
```

Deja abiertas estas ventanas:

1. Terminal en la carpeta del proyecto.
2. Navegador en `http://localhost:3000` (sin sesión iniciada).
3. Terminal secundaria para consultar PostgreSQL.
4. Explorador de archivos con la carpeta `Kubo/`.

1. Presentación (0:00 – 0:45)

- > "Hola, soy [nombre], estudiante del Tecnólogo en Análisis y Desarrollo de
- > Software. Este es ****Kubo****, un ERP + CRM para pequeñas y medianas empresas que
- > no tienen acceso a tecnología y no quieren pagar suscripciones mensuales.
- > Se instala en el propio local, en un computador pequeño o en un servidor
- > económico, y funciona incluso sin internet."

Muestra la terminal:

```
make up
```

- > "Un solo comando levanta todo el sistema: cuatro microservicios, un API

- > Gateway, la aplicación web y cuatro servicios de datos. Cada microservicio
- > está escrito en un lenguaje distinto y tiene su propia base de datos."

2. Arquitectura (0:45 – 1:45)

Muestra `kubo-docs/01-arquitectura.md` (o el diagrama renderizado).

- > "El navegador solo habla con el API Gateway. El gateway valida el token JWT
- > contra el JWKS del servicio de identidad y enruta cada petición.
- >
- > - **kubo-iam** en Java con Spring Boot: identidad, roles y auditoría.
- > - **kubo-crm** en Ruby on Rails: clientes, con los datos personales cifrados.
- > - **kubo-erp** en Elixir con Phoenix: catálogo, inventario y ventas.
- > - **kubo-analytics** en Python con FastAPI: el tablero, sobre MongoDB.
- >
- > El ERP publica un evento cuando se registra una venta, y analítica lo consume
- > por RabbitMQ. La caja nunca espera al tablero."

Muestra la carpeta raíz con los 9 repositorios:

```
ls -d kubo-*/
```

- > "Nueve repositorios: uno por cada parte del software, más la infraestructura y
- > la documentación."

3. Inicio de sesión (1:45 – 2:15)

En el navegador, en `http://localhost:3000`:

1. Muestra la pantalla de ingreso con la marca.
2. Ingresa con `admin@kubo.local` / `Admin123!`.
3. Menciona: **"La contraseña se guarda con BCrypt y el token se firma con RSA."**

4. Tablero (2:15 – 3:00)

- > "Este es el tablero del negocio. Todos estos indicadores vienen de MongoDB,
- > contruidos a partir de los eventos de venta: ventas de hoy, ingreso
- > acumulado, ticket promedio, unidades vendidas, la serie de los últimos 14
- > días, los productos más vendidos y los medios de pago."

Señala la tarjeta **Ventas de hoy** y la gráfica.

- > "Hace un momento no había nada aquí; estos datos llegaron solos desde el ERP."

5. Clientes y cifrado (3:00 – 4:00)

1. Entra a **Clientes**.
2. Señala que el documento aparece enmascarado: `\*\*\*\*\*636`.
3. Abre un cliente y muestra que el detalle sí trae el documento completo.
4. En la terminal secundaria:

```
docker exec kubo-postgres psql -U kubo_root -d kubo_crm -c \  
"select name, left(document_number_encrypted, 28) as cifrado, left(document_number_bidx
```

- > "En la base de datos el documento no se puede leer: está cifrado con
- > AES-256-GCM, que además detecta cualquier manipulación. La columna de al lado
- > es un índice ciego con HMAC: permite buscar por documento exacto sin
- > descifrarlo. En el listado mostramos solo los últimos dígitos."

6. Productos e inventario (4:00 – 4:30)

1. Entra a **Productos**: muestra precio, IVA, stock y la alerta de stock bajo.
2. Usa el botón de inventario en un producto y registra una entrada:

Tipo: **Entrada (compra)**, cantidad `10`, motivo `Compra a proveedor`.

3. Menciona: **"Cada movimiento queda en el kardex con fecha, usuario y motivo."**

7. Venta en el POS (4:30 – 5:15)

1. Entra a **Vender**.

2. Busca un producto, agrégalo dos veces, agrega otro.

3. Selecciona el cliente **Panadería El Trigo** y el medio de pago **Efectivo**.

4. Muestra el total y el IVA incluido desagregado.

5. Presiona **Cobrar**.

> "La venta se registró como V-00000X. El inventario se descontó en la misma transacción y el evento ya viajó a analítica."

6. Vuelve al **Tablero**: el indicador de hoy ya cambió.

8. Funcionamiento sin internet (5:15 – 5:50)

1. En el navegador, activa el **modo avión** (o desconecta el Wi-Fi).

2. Señala la etiqueta **Sin internet** en la barra superior.

3. Registra otra venta en el POS.

> "Sin conexión la venta no se pierde: queda en una cola local del navegador."

4. Reactiva la conexión.

> "Al volver la conexión, Kubo la envía sola y el tablero se actualiza."

5. Muestra el botón **Sincronizar** o la etiqueta de pendientes desapareciendo.

9. Cierre (5:50 – 6:30)

Muestra la documentación:

> "La documentación incluye arquitectura con diagramas C4, modelo de datos,
> contrato de la API, seguridad, despliegue, plan de pruebas, matriz de
> trazabilidad y ocho decisiones de arquitectura registradas como ADR.
>
> Kubo se instala con un comando, corre en menos de 2 GB de memoria y está
> pensado para que un negocio de barrio digitalice sus ventas sin pagar
> suscripciones. Gracias."

Frases de respaldo (por si algo falla en vivo)

| Situación | Qué decir | | --- | --- | | Un contenedor no responde | "Reviso la sonda de salud del servicio..." y ejecuta ``make ps`` | | El tablero tarda | "La consistencia es eventual: el evento viaja por RabbitMQ" | | Falla el modo offline | Muestra la cola en ``IndexedDB`` desde las herramientas del navegador | | Falla la demo por completo | Ejecuta ``make smoke`` y muestra las 33 comprobaciones en verde |

Evidencia complementaria

```
make smoke 2>&1 | tee evidencia-smoke.txt
docker compose -f kubo-infra/docker-compose.yml ps
docker stats --no-stream
```

ADR-0001 — Microservicios con monolito modular interno

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada | | Decisores | Arquitectura Kubo |

Contexto

Kubo debe cubrir cuatro dominios bien distintos (identidad, clientes, operación comercial e indicadores) y, al mismo tiempo, poder instalarse en el local de una

PYME con recursos limitados (2 vCPU / 4 GB de RAM). El equipo es pequeño y el tiempo de entrega, corto.

Decisión

Se despliegan **cinco servicios independientes** (identidad, CRM, ERP, analítica, documentos) más un API Gateway, cada uno con su propia base de datos.

Dentro de cada servicio no se subdivide en más servicios: se aplica arquitectura limpia en capas (`domain`, `application`, `infrastructure`, `presentation`) y los subdominios conviven como módulos del mismo despliegue.

Alternativas consideradas

1. **Monolito único**: más simple de operar, pero incumple el requisito de microservicios y no demuestra aislamiento por dominio.
2. **Microservicios finos** (catálogo, inventario y ventas por separado): el costo de coordinación (transacciones, consistencia, red) supera el beneficio en un equipo de una persona.
3. **Microservicios con monolito modular interno** (elegida): cada servicio puede partirse más adelante sin reescribir la lógica, porque las fronteras entre módulos ya están trazadas.

Consecuencias

- **Positivas**: se cumple el requisito, el consumo de memoria es acotado (~1.5 GB para todo el sistema), y el dominio queda listo para escalar horizontalmente el servicio que lo necesite.
- **Negativas**: no hay transacciones distribuidas; la consistencia entre servicios es eventual y se resuelve con eventos (ver ADR-0005).
- **Regla derivada**: si dos módulos del mismo servicio se comunican con demasiada frecuencia, se refactorizan a un solo módulo; si un módulo cambia

por razones distintas a su servicio, se evalúa extraerlo.

ADR-0002 — Polyrepo con contratos como fuente de verdad

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

El requisito del proyecto exige un repositorio separado por cada parte del software. Además, cada servicio usa un lenguaje y un ecosistema distinto, con herramientas de construcción incompatibles entre sí.

Decisión

****Un repositorio por componente**** (nueve en total): gateway, cuatro servicios, PWA, infraestructura, documentación y el repositorio raíz de workspace. La comunicación entre repos se gobierna por ****contratos explícitos****:

- HTTP: OpenAPI 3.1 (documentado en `kubo-docs/03-api.md`).
- Eventos: sobre estable versionado (`event\_id`, `event\_type`, `version`, `occurred\_at`, `tenant\_id`, `data`).
- Base de datos: cada servicio es dueño exclusivo de su esquema; ****ningún servicio consulta la base de otro****.

Consecuencias

- ****Positivas****: cada servicio se construye, versiona y despliega por separado; un fallo de toolchain en un lenguaje no bloquea a los demás; los equipos pueden crecer sin fricción de monorepo.
- ****Negativas****: los cambios de contrato exigen coordinación entre repos. Se mitiga con versionado semántico del contrato y una prueba de humo que valida el flujo completo antes de cada entrega.

- **Regla derivada**: un cambio incompatible en un contrato obliga a subir la versión del evento o del endpoint, nunca a modificar el significado del existente.

ADR-0003 — Multi-tenancy híbrido: `tenant\_id` hoy, RLS lista para activar

| Campo | Valor | | -- | -- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

Kubo se vende como software autoalojable: cada negocio instala su propia copia y no paga suscripción. Sin embargo, el mercado exige más adelante una versión en la nube con varias empresas en un mismo despliegue. Cambiar de modelo después es costosísimo si el modelo de datos no lo previó.

Decisión

Modelo híbrido: el código y el esquema son multi-tenant desde el día uno (todas las tablas de negocio llevan `tenant\_id` y todas las consultas filtran por él), pero **el despliegue por defecto es de un solo negocio por instalación**. La política de *row level security* de PostgreSQL está escrita y versionada en cada servicio (`db/rls/enable-rls.sql`), lista para activarse cuando exista el interceptor de transacción que fija `app.tenant\_id`.

Por qué no se activó RLS todavía

Activar RLS sin fijar la variable de sesión en cada transacción deja las consultas sin resultados: sería un fallo silencioso. Hacerlo bien requiere un interceptor que ejecute `SET LOCAL app.tenant\_id` al abrir cada transacción, probado con concurrencia. En el MVP se prioriza el aislamiento en la capa de aplicación (verificado por la prueba de humo: un segundo negocio no ve datos del

primero) y se deja la segunda barrera preparada.

Consecuencias

- **Positivas**: el mismo binario sirve para autoalojamiento y para nube; no hay

que migrar el esquema para cambiar de modelo de negocio.

- **Negativas**: hoy el aislamiento depende de la disciplina del código. Se

mitiga con la prueba automática de aislamiento y con la política SQL ya

escrita.

- **Siguiente paso**: interceptor de transacción + activación de RLS en la fase 2.

ADR-0004 — Poliglotismo deliberado: un lenguaje por contexto

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

El proyecto exige al menos cuatro microservicios con un mínimo de tres

tecnologías distintas. La tentación es elegir un lenguaje y repetirlo por

comodidad, pero eso convierte el requisito en un ejercicio vacío.

Decisión

Cada servicio se implementa en el lenguaje cuya **forma natural** encaja con su

problema, y esa elección se documenta:

| Servicio | Lenguaje | Por qué ese y no otro | | --- | --- | --- | | Identidad | Java 21 + Spring Boot 4 | El ecosistema de seguridad más maduro: Spring Security, validación, JPA, migraciones. Aquí la corrección importa más que la novedad. | | CRM | Ruby 3.4 + Rails 8 | Velocidad de evolución del modelo comercial. ActiveRecord y las migraciones permiten iterar el esquema de clientes en minutos. | | ERP | Elixir 1.17 + Phoenix 1.8 | La venta es la operación crítica: concurrencia masiva, latencia predecible y tolerancia a fallos con árboles de supervisión de OTP. | | Analítica | Python 3.13 + FastAPI | Agregaciones y, en la fase 2, pronóstico de demanda. El ecosistema de datos y el motor de agregación de MongoDB encajan sin esquema rígido. | | Gateway | TypeScript + NestJS | Capa de borde con validación de JWT, límites de tasa y enrutamiento: el mismo lenguaje del frontend reduce

el cambio de contexto. | | PWA | React 19 + Vite | Interfaz instalable, offline-first y con tipado compartido. |

Consecuencias

- **Positivas**: el equipo aprende cinco ecosistemas reales; cada servicio usa la herramienta idónea; el requisito se cumple con sentido y no por relleno.
- **Negativas**: cinco toolchains, cinco pipelines, cinco formas de hacer pruebas. Se mitiga con un contrato común (ADR-0002), con la misma estructura de carpetas y con la prueba de humo que valida el sistema completo.
- **Regla derivada**: ningún servicio se reescribe en otro lenguaje sin un ADR que lo justifique con métricas (latencia, costo, mantenibilidad).

ADR-0005 — Eventos con RabbitMQ y publicador tolerante a fallos

| Campo | Valor | | -- | -- | | Fecha | 2026-09-23 | | Estado | Aceptada (outbox diferido a fase 2) |

Contexto

Cuando se registra una venta, varios procesos deben reaccionar: analítica proyecta el indicador, notificaciones avisa al cliente, cartera actualiza el saldo. Si el ERP llamara a cada uno por HTTP, una caída de analítica rompería la venta y el acoplamiento crecería con cada consumidor nuevo.

Decisión

1. El ERP publica **eventos de dominio** en el exchange de tipo **topic** ``kubo.events``, con sobre estable y versionado (``sale.created` v1`).
2. Los consumidores **deduplican** por ``event_id``: analítica usa un índice único en MongoDB, de modo que una reentrega del broker no cuenta la venta dos veces.
3. Los mensajes que no se pueden procesar van a una **cola de mensajes muertos**

(DLQ), nunca se pierden ni bloquean la cola principal.

4. ****La venta nunca depende del bus****: la publicación es asíncrona y tolerante a fallos. Si RabbitMQ está caído, se registra la advertencia y el negocio sigue operando.

Lo que queda pendiente y por qué

El patrón **transaccional outbox** (guardar el evento en la misma transacción de la venta y publicarlo después desde una tabla de salida) elimina la ventana en la que un evento puede perderse si el proceso muere justo después del ``commit``. En el MVP se asume esa pérdida eventual y se documenta; la implementación del outbox es el primer trabajo de la fase 2.

Consecuencias

- ****Positivas****: desacoplamiento real (analítica puede caer sin afectar la caja), reacción en segundos y camino abierto a nuevos consumidores sin tocar el ERP.
- ****Negativas****: consistencia eventual (el tablero tarda milisegundos en reflejar la venta) y, en el MVP, posibilidad de perder un evento ante una caída en el instante exacto del ``commit``.
- ****Nota técnica****: se usa ``amqp` 4.x`; la serie 3.x depende de ``rabbit_common` 3.12`, que no compila en Erlang/OTP 27.

ADR-0006 — Cifrado de datos personales con AES-256-GCM e índice ciego

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

El CRM guarda datos personales de clientes (documento, teléfono), protegidos por la Ley 1581 de 2012 (habeas data). Si alguien obtiene una copia del respaldo o

accede a la base, esos datos no deben quedar legibles. Al mismo tiempo, el negocio necesita **buscar por documento** con precisión.

Decisión

1. **Cifrado a nivel de campo** con **AES-256-GCM** (confidencialidad + autenticidad: cualquier manipulación del texto cifrado se detecta y el valor se rechaza). Formato almacenado: ``base64(iv || tag || ciphertext)``.
2. **Índice ciego** con HMAC-SHA256 sobre el valor normalizado, en una columna aparte. Permite buscar por igualdad sin descifrar y sin exponer el dato.
3. **Minimización**: los listados devuelven el valor enmascarado (``*****432``); solo el detalle de un cliente concreto revela el valor completo.
4. Las claves llegan por variable de entorno (``KUBO_FIELD_ENCRYPTION_KEY``, ``KUBO_BLIND_INDEX_KEY``) y **nunca** están en el repositorio.

Alternativas consideradas

- **Cifrar toda la base** (TDE o disco cifrado): no está disponible en PostgreSQL comunitario y no protege contra un volcado lógico.
- **Guardar el dato en claro y confiar en los permisos**: insuficiente ante un respaldo filtrado, que es el escenario real más frecuente.
- **Cifrado determinista** (mismo texto cifrado para el mismo valor): permitiría buscar, pero filtra información por comparación. Se prefiere el índice ciego.

Consecuencias

- **Positivas**: un volcado de la tabla no revela documentos ni teléfonos; el GCM detecta manipulaciones; la búsqueda sigue siendo exacta y rápida.
- **Negativas**: no se puede buscar por coincidencia parcial sobre el campo cifrado (solo por igualdad) y rotar la clave exige re-cifrar los registros.
- **Siguiente paso**: envoltura de claves (KEK/DEK por negocio) con un KMS para

la versión en la nube.

ADR-0007 — JWT RS256 con JWKS: la llave privada no sale del servicio de identidad

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

Varios servicios necesitan saber quién es el usuario y a qué negocio pertenece.

La opción cómoda es que cada servicio comparta un secreto simétrico y verifique los tokens por su cuenta, pero eso reparte el secreto por todo el sistema: quien comprometa un servicio puede **emitir** tokens válidos para cualquier usuario.

Decisión

1. El servicio de identidad genera un par de llaves **RSA** y firma los access tokens con **RS256**. La llave privada vive únicamente ahí (o llega por ``KUBO_JWT_PRIVATE_KEY`` en producción).
2. Publica la llave pública en un documento **JWKS**.
3. El **API Gateway** es el único que valida la firma, contra el JWKS, con caché y refresco automático ante rotación de llave.
4. Tras validar, el gateway **elimina cualquier cabecera `X-User-`** enviada por el cliente e inyecta la identidad real antes de reenviar la petición.
5. Los servicios internos confían en esas cabeceras porque **no son alcanzables desde fuera**: solo existen en la red privada de contenedores.

Consecuencias

- **Positivas**: comprometer un servicio de negocio no permite emitir tokens; el gateway centraliza la política de acceso; los servicios no cargan librerías de criptografía ni conocen el formato del token.

- **Negativas**: el gateway es un punto único de fallo y un salto adicional de red. Se mitiga con healthchecks, límites de memoria y la posibilidad de escalarlo horizontalmente sin estado (los límites de tasa viven en Redis).
- **Siguiente paso**: mTLS entre gateway y servicios para la versión en clúster.

ADR-0008 — Alcance declarado del MVP y recortes conscientes

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

La entrega del MVP ocurre en una jornada. Es imposible cubrir un ERP/CRM de producción completo, y fingir lo contrario sería peor que recortar con criterio.

Decisión

Se prioriza lo que **demuestra arquitectura funcionando de extremo a extremo** y se documenta lo que queda fuera, con su ruta de implementación.

Incluido y funcionando

- Cuatro microservicios + gateway + PWA, con base de datos por servicio.
- JWT RS256 con JWKS, rotación de refresh tokens con detección de reuso, contraseñas con BCrypt, auditoría encadenada.
- Cifrado AES-256-GCM de datos personales con índice ciego y enmascarado.
- Venta transaccional con bloqueo pesimista de inventario y kardex.
- Eventos de dominio por RabbitMQ con consumidor idempotente y DLQ.
- Tablero con agregaciones sobre MongoDB y PWA instalable con cola offline.
- Aislamiento por negocio verificado por prueba automática.

Fuera del MVP (con ruta definida)

| Recorte | Por qué | Cuándo | | --- | --- | --- | | *Transactional outbox* | Requiere tabla de salida y publicador de barrido; el MVP ya no bloquea la venta por el bus | Fase 2 | | Activación de RLS | Necesita interceptor de transacción probado con concurrencia | Fase 2 | | Refresh token en cookie httpOnly | Exige BFF con cookies; hoy vive en el navegador (documentado) | Fase 2 | | Facturación electrónica DIAN (UBL 2.1, CUFE, QR) | Requiere ser Proveedor Tecnológico autorizado; el puerto de facturación queda preparado | Fase 3 | | Multi-rubro (Vertical Packs) | El MVP demuestra un vertical (retail) completo | Fase 3 | | Kubernetes, Terraform, observabilidad completa | El despliegue objetivo es un local con Docker Compose | Fase 4 | | Recuperación de contraseña por correo | Requiere proveedor de correo y plantillas | Fase 2 |

Consecuencias

- **Positivas**: el sistema se entrega funcionando y verificado, con un mapa honesto de lo que falta; nada se presenta como terminado sin estarlo.
- **Negativas**: el producto no está listo para un negocio real sin antes completar la fase 2 (outbox, RLS, cookie httpOnly, recuperación de contraseña).
- **Regla derivada**: ningún recorte se oculta en la documentación; cada uno aparece en este ADR y en la matriz de trazabilidad.